[AG] POST /files Сохранение файлов

История изменений

Список ссылок (ссылка) на задачи в Jira. В основном: 1 метод = 1 задача = 1 Confluence.

Связанные документы (внутренние)

Тест-кейсы, описания БД и другие статитьи по проекту, которыми пользовались в ходе разработки

Оглавление:

Общее описание

Нажмите здесь, чтобы развернуть...

Работа с файлами - загрузка ДО создания сущности

Use Case (для Frontend):

Создание фильма в приложении администратора

Предусловие:

Администратор заходит на экран создания нового фильма.

Основной сценарий:

1. Администратор вводит данные о фильме и загружает картинки к нему.

В момент выбора картинок браузер хранит выбранные файлы в состоянии формы / памяти до отправки.

2. Администратор нажимает кнопку "Сохранить".

Frontend отображает лоудер пользователю.

3. **Frontend вызывает метод**

POST /file , чтобы загрузить файл картинки на сервер.

(если там не только картинки, но и трейлер, то будет несколько раз вызов, пока каждый файл не будет загружен отдельно).

4. **Frontend вызывает метод**

POST /movie

чтобы создать фильм.

id картинок и трейлеров загруженных на шаге 3 добавляются в JSON

❗ Важно: **POST /movie вызывается только после успешной загрузки всех файлов на сервер - в файловое хранилище.**

5. Frontend получает ответ HTTP 201 Created, завершает отображение лоудера и переводит администратора в список фильмов или на страницу созданного фильма.

Новый фильм отображается в интерфейсе.

Метод предназначен для загрузки и сохранения файлов в системе. Он является универсальным решением для работы с файлами, которое используется в различных сценариях платформы: загрузка резюме и справок о судимости при создании заявки на преподавание, прикрепление документов к заявкам, загрузка аватарок пользователей и других медиафайлов. Метод предоставляет единый интерфейс для сохранения файлов, поддерживая идемпотентность, что гарантирует надёжность даже при повторных отправках.

Для кого: метод доступен как авторизованным, так и неавторизованным пользователям. В зависимости от контекста использования, он может вызываться через разные API-префиксы:

- `/private/v1.0/files` — для преподавателей;
- `/public/v1.0/files` — для студентов;
- `/admin/v1.0/files` — для администраторов и модераторов платформы.

Входные данные: метод принимает JSON-объект с двумя обязательными полями:

- `name` — имя файла с расширением (например, "document.jpg"), которое будет использоваться для идентификации;
- `content` — содержимое файла в формате Base64. Это позволяет передавать файлы любых типов (изображения, PDF, документы) безопасно через JSON-формат, избегая

проблем с бинарными данными при передаче по сети.

Ожидаемый результат: при успешной загрузке файла сервер возвращает HTTP-201 с JSON-объектом, содержащим:

- `name` — исходное имя файла;
- `format` — расширение файла (например, "jpg", "pdf", "png");
- `link` — сгенерированная ссылка на сохранённый файл в файловом хранилище (ФХ).

Эта ссылка затем используется в других методах системы (например, при создании заявки на преподавание). В случае возникновения ошибок сервер возвращает соответствующий HTTP-статус с текстовым описанием проблемы.

Доступы

Метод доступен неавторизованным и авторизованным пользователям.

Ограничения

Rate Limit: не более 10 запросов в минуту с одного ip-адреса.
В случае превышения лимита возвращать HTTP-429.

Запрос

Method type: POST

BaseURL: https://u-verse.ru/api

API name:

- private
- public
- admin

API version: v1.0

Method name: /files

Authorization:

- Тип: Token (Bearer Token).
- Необязательная.

Headers:

- Content-Type: application/json
- Idempotency-Key: <Key>
- Authorization: Bearer <Token>

Пример запроса:

```
POST https://u-verse.ru/api/private/v1.0/files
```

```
POST https://u-verse.ru/api/public/v1.0/files
```

```
POST https://u-verse.ru/api/admin/v1.0/files
```

Query-parametrs:

- Нет

Тело запроса (Body)

Пример:

```
{
  "name": "document.jpg",
  "content": "JVBERi0xLjQKMSAwIG9iago8PC9UeXBlL0NhdGFsb2cvUGFnZXMgMiAwIF
}
```

Пример ответа - Успех

Код HTTP: 201

Тело ответа (Body):

```
{
  "id": "49926459-45b8-4144-8c1e-7c5dfe40dd91",
  "name": "document.jpg",
  "format": "jpg",
  "link": "https://files.u-verse.ru/7a5f76a87f6s7df6.jpg"
}
```

Пример ответа - Неверный запрос (ошибка формата)

Код HTTP: 400

Тело ответа (Body):

```
{
  "message": "Данные не прошли проверку.",
  "details": {
    "name": "Поле обязательно для заполнения.",
    "content": "Поле обязательно для заполнения."
  }
```

Пример ответа - Неверный формат

Код HTTP: 400

Тело ответа (Body):

```
{
  "message": "Некорректный формат файла. Допустимы форматы: pdf, png, jp
}
```

Пример ответа - Превышает размер

Код HTTP: 400

Тело ответа (Body):

```
{
  "message": "Размер файла превышает допустимый. Допустимые размер файла
}
```

Пример ответа - Дубликат ключа идемпотентности

Код HTTP: 409

```
{
  "message": "Вы уже использовали этот ключ идемпотентности для другого
}
```

Пример ответа - Превышение лимита запросов

Код HTTP: 429

Тело ответа (Body):

```
{
  "message": "Превышено количество запросов."
}
```

Пример ответа - Сервер недоступен

Код HTTP: 503

Тело ответа (Body):

```
{
  "message": "Сервер не доступен, попробуйте еще раз"
}
```

Требования к реализации

Основной сценарий:

1. Проверить, что с одного ip-адреса пришло не более 10 запросов за последнюю минуту:
   a. Если количество запросов превышает, то вернуть ошибку с кодом HTTP-429.
   b. Иначе - далее.
2. Проверить, была ли за последние 24 часа создан запрос с таким ключом идемпотентности и полные данные запроса:
   a. Если да, то вернуть успешный ответ с кодом HTTP-201 и ранее сохраненным результатом из БД.
   b. Если ключ идемпотентности найден, но данные по апросу не совпадают, то вернуть ошибку HTTP-409.
   c. Иначе - далее.
3. Проверить наличие обязательных полей. Если хотя бы одно обязательное поле отсутствует, то вернуть ошибку HTTP-400 с соответствующем текстом.
4. Проверить формат файла (допустимы форматы: pdf, png, jpg, jpeg.). Если формат файла некорретный, то вернуть ошибку HTTP-400 с соответсвующим текстом.
5. Проверить размер файла (не более 2 Мб). Если размер файла превышает допустимый, то вернуть ошибку HTTP-400 с соответсвующим текстом.

6. Сохранить файлы в ФХ.
7. Сохранить записи в БД.
8. Вернуть успешный ответ HTTP-201.

Обработка исключительных ситаций:

1. Внутрення ошибка - возможна на любом шаге.

В процессе работы алгоритма не удается подключиться к ФХ, БД или происходит внутрення ошибка. Вернуть ошибку HTTP-503.

2. Ошибка при создании сущности

Если в процессе создания основной сущности возникла ошибка, то созданные файлы, не связанные ни с одной сущностью, ежедневно должны удаляться фоновым процессом.

Диаграмма:

[AG] UML - Подача заявки на преподавание

Маппинг данных

Для тела запроса:

Параметр в JSON	Описание	Обяз.	Тип данных JSON	В БД
name	Наименование файла	Да	string	
content	Данные файла	Да	base64	

Для тела ответа:

Параметр в JSON	Описание	Обяз.	Тип данных JSON	В БД
id	Идентификатор файла	Да	string	file.id
name	Наименование файла	Да	string	file.name

format	Формат файла	Да	string	file.format
link	Ссылка для скачивания файла	Да	string	file.link